Pakistan AQI Predictor

Built by Syed Sameer Rizvi
Email: *ashrafsameer682@gmail.com*
Live app: ***https://pakaqipredictor.netlify.app***

The 3 day forecast of 6 cities that is predicted and forecasted hourly for the next 72 hours, from a machine learning model trained on two years of data.

The Goal

The goal was to build a serverless working air quality forecasting service that when a person opens the app and picks a Pakistani city and sees where the US EPA AQI is headed for the next 3 days. So I decided why not try to make it predict for more cities, so the app forecasted 6 cities: Karachi, Lahore, Islamabad, Faisalabad, Peshawar and Quetta.

Choosing the Data Source

To choose the data source there were 3 opinions:

- AQICN
- OpenWeather
- OpenMeteo

I chose Open-meteo as its archive dates back years and which is exactly my project needed for a two year backfill requirement.

AQICN and OpenWeather had their constraints like AQICN does not give enough data to train on and OpenWeather has history but charges for the depth needed. SO OpenMeteo gave air quality endpoints for the pollutants, its ERA5 archive for deep historical weather and its forecast endpoint for recent and future weather. The call for the plain JSON endpoints with requests library rather than the FlatBuffers client because JSON is far easier to read and debug.
One honest result of this decision, which instead of keeping quiet: OpenMeteos pollution data is not actual measurements from ground stations but a model that has been created **(the CAMS reanalysis).** Making predictions about the results from another model. That comes with its **own errors**. Any model I create cannot be more accurate than the data it is trained on. This means there is a real limit to how accurate it can be. I made this choice because I wanted to have access to a lot of historical data.

Fetching and Cleaning the Data

Two Weather Paths:

I find that OpenMeteo splits weather across two services so I fetched from both OpenMeteo services. The ERA5 archive gives history but lags real time by about five days so I used the ERA5 archive for backfill over date ranges that end at least a week in the past. The forecast endpoint covers past and future so I used the forecast endpoint, for the live pipeline and dashboard to get future weather that feeds predictions.

Cleaning and Robustness:

- **Timezone discipline**: every timestamp is stored in UTC. The only change from UTC to time at two points: when the build is the hour of day feature and show the data on the dashboard. Mixing timezones inside the data is a painful bug so I made the rule explicit.
- **Merging:** the pollutant frame and the weather frame are combined using a join based on the timestamp. This means that only rows with a matching timestamp, in both frames are kept. Any duplicate timestamps are removed.
- **Missing values:** missing results, in a column filled with NaN values instead of causing the fetch to fail. Rows that don't have all the required features or targets are removed later during feature building not quietly fixed or skipped.
- **Retry and backoff**: when there are API issues like rate limits or 5xx errors the system retries automatically. The wait time increases with each attempt: 1 second, 2 then 4 8 and 16 seconds. This helps with lived problems that resolve on their own. I made sure to exclude 429 errors from the retry loop because I found that retrying aggressively actually made rate limiting worse.
- **Chunking:** when dealing with long date ranges the data is fetched in pieces of 120 days and then combined. This prevents timeouts or reaching size limits that can happen with one large request.
- **Per city isolation:** if a fetch fails for one city it doesn't stop the others. Failures are. Reported, but not raised as errors. This keeps the process running even when one city has issues.

Feature Engineering

The Feature Groups:

- **Time:** hour, day of week day of month month and a weekend flag all based on time because pollution patterns follow the local clock. Hour and month are represented using sine and cosine pairs so that 23:00 and 00:00 are treated as close to each other, not as ends.

- **Lags:** AQI and PM2.5 values at 1 2 3 6 12 24 48, 72 and 168 hours ago (up to one week back).
- **Rolling stats**: standard deviation of AQI over the last 6, 12, 24 and 72 hours, plus the minimum and maximum AQI over the last 24 hours.
- **Rate of change**: difference in AQI over the 1, 3 6 and 24 hours to capture how quickly pollution is rising or falling.
- **Weather dispersion**: wind speed and direction transformed into u and v components because wind direction is treated 0 degrees and 360 degrees as the same. Also include temperature to dew point spread and a stagnation index, which combines pressure, with low wind speed. These features help estimate how well pollution disperses.
- **Interaction:** hour of day multiplied by wind speed since emissions often follow schedules and how pollution spreads depends on wind at that time.

During backfill the future weather inputs use weather. But In production the future weather inputs use forecast weather, which is less accurate. Offline metrics therefore are slightly optimistic compared with performance. Observing these differences shows that the design is worth stating rather than hiding.

Feature Store

Features live in Hopsworks Serverless are on the free tier, which influenced several decisions I wouldn't have made otherwise. Documented them because they reflect engineering choices shaped by real limitations.

- Reads go through the store. The offline read path (Arrow Flight) was unstable on the free tier and kept failing with server side errors while the online path worked consistently. So I read online=True. Writes still update both stores. It relies only on the online path for reading.
- HUDI streaming write path. The alternative Delta write path failed on Windows because it requires Linux components. So it uses HUDI with stream enabled, which is the base of the Kafka approach that runs well in my setup.
- `Wait_for_job` is disabled on writes. The writer sends data to the store immediately but the background offline materialization job fails on the free tier. Waiting for it would cause delays. Fail with errors even though the data is already safely stored and readable.
- Composite primary key `(city_id timestamp)`. An insert for one city and one hour updates that row. This means hourly upserts run cleanly and one city can never overwrite another city's data.
- A separate lightweight serving feature group. For predictions, just keep only one updated row per city using `city_id` as the key and read it through a feature view. This avoids

loading the Hopsworks engine, which would exceed the 512MB memory limit, on the free backend host.

Model Experiments and Comparison

I did a comparison between eight methods that included simple starting points, traditional machine learning, gradient boosting, a statistical model and deep learning. The models were evaluated city by city. For each time period using a chronological hold out split. Every actual model was tested against persistence, which means tomorrow is the same as today. This is because beating that starting point is the only way a model can show it has learned something more, than just repeating what happened before.

The models I tried:

- Baselines: persistence and seasonal naiveness (same hour last week).
- Classical ML: ridge regression and random forest.
- Gradient boosting: XGBoost, untuned and tuned by grid search on validation RMSE.
- Statistical: SARIMAX with daily seasonality and weather as exogenous inputs.
- Deep learning: a small two layer LSTM predicting all three horizons at once.

The table below shows the best model for each city at the 24 hour horizon, from the final comparison. R2 is the share of variance explained; higher is better.

City	Best model (24h)	R2	RMSE	MAE
Islamabad	**XGBoost tuned**	**0.76**	**16.5**	**12.8**
Peshawar	**XGBoost tuned**	**0.63**	**19.1**	**14.7**
Karachi	**XGBoost**	**0.52**	**8.1**	**5.8**
Lahore	**Ridge**	**0.44**	**28.6**	**18.9**
Faisalabad	**XGBoost**	**0.30**	**21.6**	**16.5**
Quetta	**XGBoost**	**0.20**	**30.6**	**22.2**

First the fancy models were lost. At the 72 hour horizon on Lahore, the LSTM and SARIMAX both had bad R2 scores (around minus 4) much worse than persistence because they overfit and were not strong for long range predictions. Second, no single model wins every cell. **Tuned XGBoost** wins the cities but plain XGBoost and even ridge beat it out on the noisier cities, where there is less signal for tuning to use.

Pooled vs City

Also tested if training one pooled model across all cities performs better than training a model for each city. The pooled model won everywhere. The per city models only helped with Karachi for longer time horizons. Either hurt or matched performance elsewhere. This is important because it supports using a model that works well across cities instead of having to maintain six separate models.

The Chosen One and Why?

I used a tuned XGBoost model for each forecast horizon, three models in total covering 24, 48 and 72 hours. Here is why:

- The model outperforms the persistence baseline in cities where there is predictive signal. That's important because it shows the model is actually forecasting and not just repeating the value.
- It performs consistently across all horizons. Unlike LSTM and SARIMAX which broke down at ranges this model stays stable and reliable.
- In our tests the pooled model beat the per city models. So using one model across cities is simpler to run and maintains accuracy without loss.
- Tuning helped a bit for Lahore and Peshawar at the middle and longer horizons. The improvement was modest but meaningful so I kept the version.

I didn't go for a per city per horizon model. An approach could squeeze out a little accuracy here and there but it would mean running and maintaining many different models. That would be messy. For a service that needs to stay active and retrain regularly one dependable pooled model is the engineering choice. The results show I lose very little in accuracy.

Why not R^2 of 0.7 everywhere?

My 24 hour R2 varies from 0.76 in Islamabad to 0.20 in Quetta. This spread is not a mistake; it shows how difficult each city really is to forecast. I preferred to explain this rather than conceal the weaker cities.

- Islamabad forecasts best because it is a greener city with regular daily pollution cycles and lower extreme variance. Regular patterns are easy to learn. Which makes sense by the way.
- Karachi is harder to forecast because it is a city. Sea breezes and shifting air make its pollution more driven by weather and irregular so there is less repeatable structure to learn.
- Lahore and Faisalabad experience extreme random events: winter smog and crop residue burning create spikes that depend on burning schedules and regional transport, which the

weather features cannot fully capture. The model captures the baseline but not the event driven tails, which limits its 24 hour R2.

- Quetta has a signal. Its arid climate pollution shows spikes and thinner data so there is simply less structure to model.

Beyond the difficulty of each city two factors keep every city from a score. The target itself is a modeled product (CAMS) that has its noise. Forecasting 48 and 72 hours ahead involves irreducible uncertainty because future AQI depends on future emissions and weather that cannot be known perfectly in advance. Given all this an honest 24 hour R2 near 0.7 on the cities is a genuinely good result. A model claiming 0.9, across all cities and horizons would raise suspicion of leakage.

Serving, Deployment and Automation

The trained model bundle is served by a FastAPI backend hosted on Render. This backend loads the model bundle, retrieves the serving vector from the feature store, generates a forecast and returns it along with a plain language summary written by a Groq model and recent news about Pakistan air quality from GNews. A React and TypeScript frontend hosted on Netlify displays the forecast chart, horizon cards, alerts and a panel showing the SHAP feature importances that explain each prediction.

SHAP explanations

The SHAP computes values once using a TreeExplainer and includes them in the model bundle for each forecast horizon. This means the dashboard can display which features caused each prediction to go up or down without needing to do any computation. It turns the forecast from a box into something users can actually explore and understand.

Two schedulers

- A feature pipeline runs every hour on GitHub Actions. It fetches data, computes new features and updates both the feature store and the serving vectors so predictions remain current.
- A training pipeline runs weekly. It retrains the model behind a metrics gate. The new model is only promoted if its 24 hour and 48 hour R2 scores do not drop compared to the deployed model. If a retrain performs worse it is. The existing model stays in place. Promotion of a reviewed model to the service requires a manual step. An unreviewed model never goes live on its own. I made this choice deliberately. My own testing showed that a retrain can sometimes perform worse and relying solely on metrics to deploy a model to production is risky.

Blockers I hit and how I fixed them

Infrastructure and platform

- **Render free tier memory:** The full Hopsworks batch engine launches the HSFS Python engine and exceeds the 512MB limit on Render. Isolved this by adding a lightweight serving feature group, with one overwritten row per city read through a feature view. Thus serving never loads the engine.
- **Hopsworks offline read broken:** The offline Arrow Flight read failed repeatedly on the tier. I switched all reads to the store, which proved reliable at our scale.
- Delta write path on Windows. Delta write path on Windows failed because it expects Linux only components. I switched to the HUDI streaming write path, which works on Windows.
- **Hopsworks lock timeouts:** Under contention the online store occasionally returned a MySQL lock wait timeout and the background materialization job sometimes dropped connection mid launch. The serving write still succeeded so I treated these as transient. Let the next run recover.

Dependencies and environment

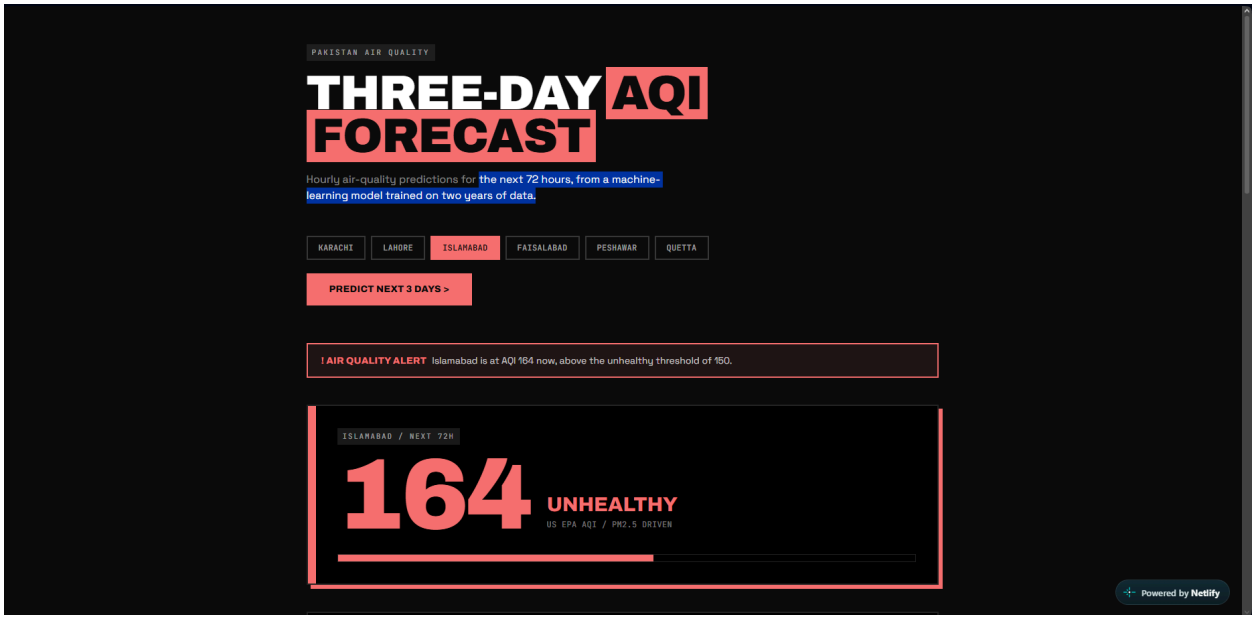
- **Version pinning:** Newer numpy, scipy and scikit-learn broke sklearn imports so I pinned numpy 2.1.3 scipy 1.14.1 and scikit-learn 1.5.2 as the known set. A later install bumped numpy. Broke the compiled ABI, which I fixed by force reinstalling the pinned trio from prebuilt wheels.
- **OpenMeteo retry trap:** Including 429 in the retry list multiplied every rate limited call by the retry count. I removed it from the retry set.
- **Groq model deprecation:** The model I first used was deprecated so I moved to an one and raised the token limit because the newer model spends tokens reasoning before it answers.

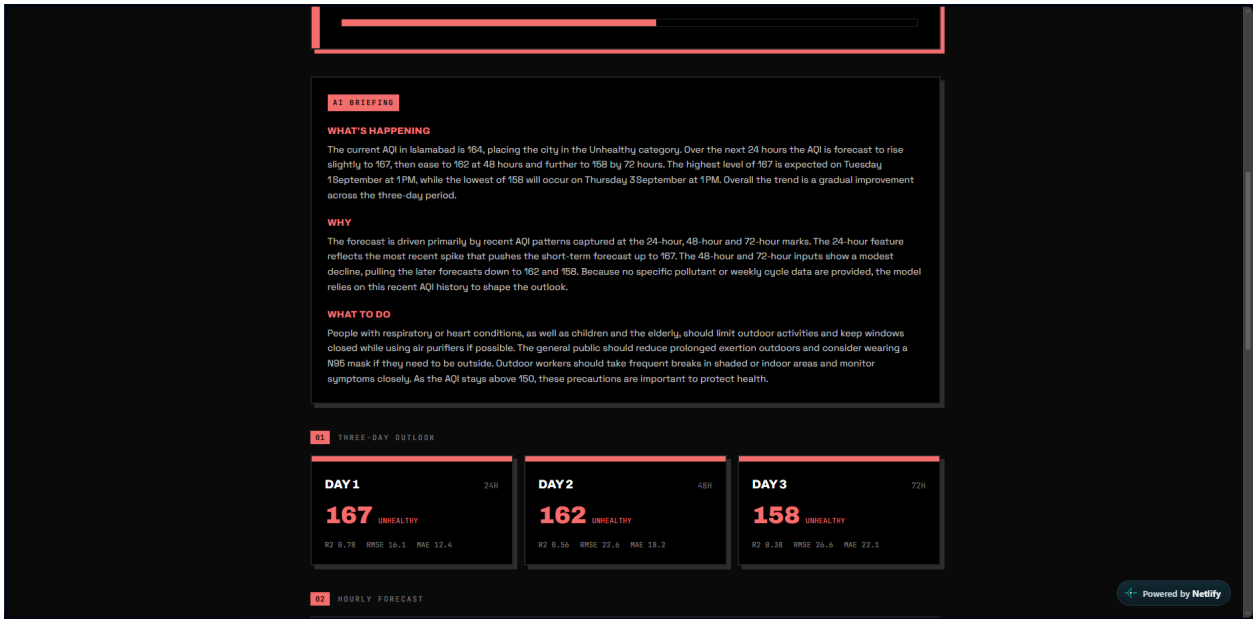
Deployment and version control

- **Silent staging failures:** On one deploy only the frontend file was committed while the backend and model bundle were left unstaged. The live site rendered a panel with no data behind it. I traced it by checking the commit contents and the git status rather than guessing.
- **A workflow that never ran:** The training pipeline workflow failed on every push, with no jobs executed. The cause was a committed workflow file: the content was edited locally but never actually committed so GitHub was running an empty file. Committing the content fixed it and the run went green.

- **A 500 that was really a 404**: A forecast request returned a 500 that turned out to be a city id: the ids are prefixed (pk-karachi not karachi) and the plain id raised a KeyError. Using the prefixed id fixed it.

App Screenshots





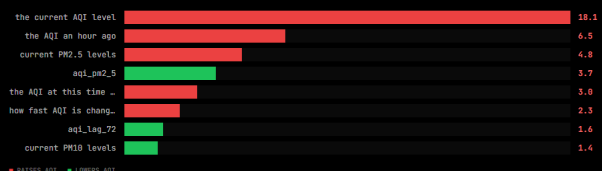
02 HOURLY FORECAST

03 WHAT'S DRIVING THIS

24H 48H 72H

WHAT IS DRIVING THIS

SHAP feature importance for the 24h forecast



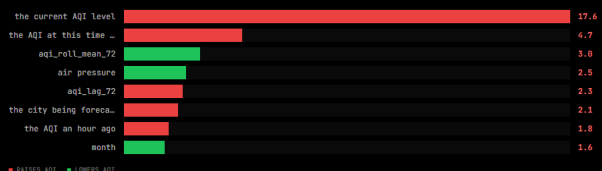
Powered by Netlify

03 WHAT'S DRIVING THIS

24H 48H 72H

WHAT IS DRIVING THIS

SHAP feature importance for the 48h forecast



04 IN THE NEWS

PAKISTAN AIR QUALITY NEWS

Pakistan significantly affects NCR, Punjab's annual PM2.5 concentration: Report

THE ECONOMIC TIMES / 8/26/2026

BUILT BY SYED SAMEER RIZVI

ashrafsameer682@gmail.com

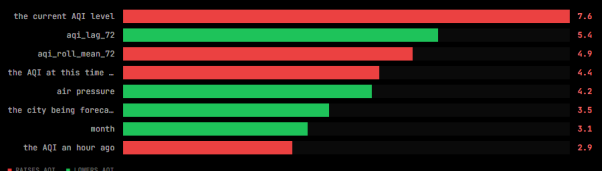
Powered by Netlify

03 WHAT'S DRIVING THIS

24H 48H 72H

WHAT IS DRIVING THIS

SHAP feature importance for the 72h forecast



Limitations that needs to be addressed here

Longer time periods are not good: The accuracy for 48 hours and 72 hours. Becomes negative for the most difficult cities. This is normal for air quality predictions at that distance. I show the real numbers instead of hiding them.

The goal is based on a model, not on measurements: Since the OpenMeteo pollution data comes from CAMS we are predicting what the model says, which limits how accurate my own predictions can be. If there is data from ground stations that would make the results better.

Offline measurements are a bit too positive: The backfill process uses weather data while the live system uses forecast weather so the real performance will be a little lower than the test numbers we show.

Free plan limits: The online store has a limit on the number of rows. The backend has a limit on memory. A bigger set of cities would need a mixed approach using the online store for recent data and a Parquet file of full history, for training.

Changing models is done by hand on purpose: Retraining is done automatically and controlled. Putting a checked model into the live system is a manual action. For a system this would grow into monitoring testing new versions carefully and automatic fixes if something goes wrong.